

# Zadanie 1 — wizytówka, krok po kroku

Zaczynasz od pustego katalogu i dochodzisz do działającej wizytówki z formularzem, który sprawdza dane i pokazuje błędy pod polami.

Każdy fragment kodu w tej instrukcji jest **dokładnie taki**, jak w gotowym rozwiązaniu — dopisujesz je po kolei i nic nie musisz zgadywać.

**Nie przepisuj wszystkiego na raz.** Rób krok po kroku i po każdym otwórz stronę w przeglądarce. Do katalogu mb2/zad1/ zaglądaj wtedy, gdy coś nie działa i nie wiesz dlaczego.

---

## Krok 0 — przygotowanie

```
git switch main
git pull
git switch -c inf04-mb02
mkdir -p zadania/mb02
```

Utwórz trzy puste pliki: `wizytowka.html`, `style.css`, `main.js`.

Potrzebny będzie jeszcze awatar. Możesz skopiować gotowy `avatar.svg`, wstawić własne zdjęcie albo dowolny obrazek z dysku.

Stronę otwierasz **podwójnym kliknięciem** w `wizytowka.html` — to zwykły plik HTML, serwer nie jest potrzebny. Po każdej zmianie odświeżasz F5.

---

## Część 1 — HTML

### Krok 1 — szkielet strony

Cała zawartość `wizytowka.html` na start:

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
```

```
scale=1.0">
  <title>Wizytówka – Jan Kowalski</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="naglowek"></header>

  <main></main>

  <footer class="stopka"></footer>

  <script src="main.js"></script>
</body>
</html>
```

Trzy rzeczy, które muszą tu być:

| Element                | Po co                                      |
|------------------------|--------------------------------------------|
| lang="pl"              | czytnik ekranu wie, w jakim języku czytać  |
| <meta charset="UTF-8"> | bez tego polskie znaki wyjdą jako krzaczki |
| <meta name="viewport"> | strona nie będzie zmniejszona na telefonie |

<script> idzie **na końcu** `<body>`, nie w <head>. Gdyby był w nagłówku, JavaScript wykonałby się, zanim powstaną elementy strony, i `querySelector` zwróciłby `null`.

**Sprawdź:** otwórz stronę — powinna być pusta i biała, bez błędów w konsoli (F12 → zakładka *Console*).

---

## Krok 2 — nagłówek z awatarem

Wypełniasz <header class="naglowek"> z kroku 1:

```
<header class="naglowek">
  
```

```

<div class="naglowek__tekst">
  <h1>Jan Kowalski</h1>
  <p class="podtytul">Technik programista · klasa 4</p>
</div>
</header>

```

Wpisz **swoje** imię i klasę — to Twoja wizytówka, nie Jana Kowalskiego.

`<div class="naglowek__tekst">` jest potrzebny, bo nagłówek będzie flexboksem: awatar to jeden element, a cały tekst obok — drugi. Bez tego opakowania `<h1>` i `<p>` ustawiłyby się obok awatara jako dwie osobne kolumny.

**O atrybucie `alt`.** To nie jest podpis pod obrazkiem — to opis dla kogoś, kto obrazka nie widzi. Zasada postrzegalności z WCAG, o którą pytają pytania 11 i 372 z rozgrzewki.

| Źle                           | Dlaczego                                           |
|-------------------------------|----------------------------------------------------|
| <code>alt="obrazek"</code>    | nic nie wnosi                                      |
| <code>alt="avatar.svg"</code> | nazwa pliku to nie opis                            |
| <code>alt=""</code>           | dopuszczalne <b>tylko</b> dla dekoracji bez treści |

**Sprawdź:** zepsuj na chwilę ścieżkę w `src` — w miejscu obrazka powinien pojawić się Twój tekst z `alt`.

---

## Krok 3 — sekcje treści

Wypełniasz `<main>`:

```

<main>

  <section id="o-mnie">
    <h2>O mnie</h2>
    <p>
      Uczę się programowania aplikacji webowych. Najbardziej
      interesuje mnie
      front-end, ale chcę rozumieć też to, co dzieje się po stronie
      serwera.
    </p>
  </section>

```

```
<section id="umiejetnosci">
  <h2>Umiejętności</h2>
  <ul class="lista-umiejetnosci">
    <li>HTML – struktura semantyczna, formularze</li>
    <li>CSS – selektory, box model, flexbox</li>
    <li>JavaScript – DOM, zdarzenia, ES6</li>
    <li>Git – gałęzie, Pull Request</li>
  </ul>
</section>

<section id="kontakt">
  <h2>Kontakt</h2>
</section>

</main>
```

Sekcja #kontakt na razie zawiera sam nagłówek — formularz wstawimy do niej w następnym kroku.

**Kolejność nagłówków ma znaczenie.** Na stronie jest jedno <h1>, a sekcje zaczynają się od <h2>. Nie wolno przeskakiwać z <h1> na <h3> tylko dlatego, że „<h3> jest mniejsze” — od rozmiaru jest CSS.

Wypełnij też stopkę:

```
<footer class="stopka">
  <p>
    Jan Kowalski ·
    <a href="https://github.com/jan-kowalski">GitHub</a> ·
    <a
href="mailto:jan.kowalski@example.com">jan.kowalski@example.com</a>
  </p>
</footer>
```

---

## Krok 4 — formularz

Cały formularz wstawiasz **do sekcji #kontakt**, pod nagłówkiem <h2>:

```
<section id="kontakt">
  <h2>Kontakt</h2>

  <form id="formularz" novalidate>

    <div class="pole">
      <label for="imie">Imię i nazwisko</label>
      <input type="text" id="imie" name="imie">
```

```

    <p class="blad" id="blad-imie"></p>
</div>

<div class="pole">
    <label for="email">Adres e-mail</label>
    <input type="email" id="email" name="email">
    <p class="blad" id="blad-email"></p>
</div>

<div class="pole">
    <label for="temat">Temat</label>
    <select id="temat" name="temat">
        <option value="">– wybierz temat –</option>
        <option value="wspolpraca">Propozycja współpracy</option>
        <option value="pytanie">Pytanie o projekt</option>
        <option value="inne">Inne</option>
    </select>
    <p class="blad" id="blad-temat"></p>
</div>

<div class="pole">
    <label for="wiadomosc">Wiadomość</label>
    <textarea id="wiadomosc" name="wiadomosc" rows="5"></textarea>
    <p class="blad" id="blad-wiadomosc"></p>
</div>

    <button type="submit" class="przycisk">Wyślij wiadomość</button>
</form>

</section>

```

## Wzorzec, który powtarza się przy każdym polu

|                                   |                               |
|-----------------------------------|-------------------------------|
| <div class="pole">                | ← opakowanie, do niego dodamy |
| klasę błędu                       |                               |
| <label for="ID">...</label>       | ← etykieta wskazująca na pole |
| <input id="ID">                   | ← pole (albo select, albo     |
| textarea)                         |                               |
| <p class="blad" id="blad-ID"></p> | ← puste miejsce na komunikat  |
| </div>                            |                               |

Trzy identyfikatory muszą do siebie pasować: for="imie", id="imie" i id="blad-imie". JavaScript znajdzie miejsce na komunikat, doklejając blad- do id pola — jeśli nazwiesz je inaczej, przestanie działać.

**Po co `novalidate`?** Bez niego przeglądarka pokaże własne dymki i nie dojdzie do naszego kodu. Wyłączamy je, bo wymaganie mówi: komunikat **pod polem**.

**Po co pusta pierwsza opcja w `<select>`?**

```
<option value="">– wybierz temat –</option>
```

Bez niej lista od razu pokazuje „Propozycja współpracy” i nie da się odróżnić świadomego wyboru od braku wyboru. Pusta wartość sprawia, że w JavaScriptcie „nic nie wybrano” to po prostu "" — tak samo jak puste pole tekstowe.

Trzy realne tematy plus pozycja startowa — razem cztery `<option>`.

**Sprawdź:** kliknij w napis „Imię i nazwisko” — kursor musi wskoczyć do pola. Jeśli nie wskakuje, `for` nie zgadza się z `id`.

---

## Krok 5 — miejsce na podziękowanie

Dopisujesz **pod formularzem, wewnątrz sekcji `#kontakt`**:

```
<div id="podziekowanie" class="podziekowanie" hidden>
  <h3>Dziękuję za wiadomość!</h3>
  <p id="podziekowanie-tresc"></p>
</div>
```

Atrybut `hidden` chowa element od startu. Po poprawnym wysłaniu formularza zamienimy je miejscami:

```
formularz.hidden = true;      // schowaj formularz
podziekowanie.hidden = false; // pokaż podziękowanie
```

`<p id="podziekowanie-tresc">` jest puste — tekst wstawi JavaScript, bo zawiera imię i wybrany temat.

**Sprawdź:** strona wygląda jak wcześniej — podziękowania nie widać. Jeśli widać, brakuje `hidden`.

Na tym HTML jest skończony. Zacommituj:

```
git add zadania/mb02/wizytowka.html
git commit -m "Wizytówka: struktura HTML i formularz kontaktowy"
```

---

## Część 2 — CSS

Cały ten rozdział to plik `style.css`. Dopisujesz bloki po kolei, od góry.

## Krok 6 — zmienne, reset i szerokość strony

```
:root {
  --kolor-glowny: #5B3FD6;      /* rgb(91, 63, 214) */
  --kolor-ciemny: #40299B;     /* rgb(64, 41, 155) */
  --kolor-blad: #C0392B;       /* rgb(192, 57, 43) */
  --kolor-tlo: #F4F4F7;
  --kolor-tekst: #1E1E24;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  line-height: 1.6;
  color: var(--kolor-tekst);
  background: var(--kolor-tlo);
}

main, .stopka {
  max-width: 720px;
  margin: 0 auto;
  padding: 0 1.5rem;
}
```

**Wybierz własny kolor przewodni** i przelicz go ręcznie na RGB — dokładnie tak, jak w pytaniu 3 z rozgrzewki. Przeliczenie zapisz w komentarzu na początku pliku:

```
/* Kolor przewodni: #5B3FD6
   0x5B = 5·16 + 11 = 91
   0x3F = 3·16 + 15 = 63
   0xD6 = 13·16 + 6 = 214
   czyli rgb(91, 63, 214) */
```

Litery to A=10, B=11, C=12, D=13, E=14, F=15.

\* { box-sizing: border-box; } sprawia, że width: 300px to **cała** szerokość razem z paddingiem i obramowaniem. Ustaw to raz i zapomnij — inaczej każdy padding będzie rozpychał elementy poza zaplanowany rozmiar.

margin: 0 auto przy ustalonej max-width wyśrodkowuje blok na stronie.

**Sprawdź:** tło zrobiło się jasnoszare, tekst nie dotyka krawędzi okna.

---

## Krok 7 — nagłówek na flexboksie

```
.naglowek {
  display: flex;
  align-items: center;
  gap: 1.5rem;

  max-width: 720px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
}

.awatar {
  width: 96px;
  height: 96px;
  border-radius: 50%;
  flex-shrink: 0;          /* awatar nie kurczy się na wąskim
ekranie */
}

.naglowek h1 {
  margin: 0;
  font-size: 1.9rem;
  color: var(--kolor-glowny);
}

.podtytul {
  margin: 0.25rem 0 0;
  color: #5A5A66;
}
```

`align-items: center` wyrównuje awatar i tekst w pionie, `gap` daje odstęp między nimi — bez marginesów na elementach.

`flex-shrink: 0` to drobiazg, o którym łatwo zapomnieć: bez niego przy wąskim oknie flexbox ściśnie okrągły awatar w owal.

**Sprawdź:** awatar i tekst stoją obok siebie. Zwężaj okno — awatar ma zostać kołem.

---

## Krok 8 — sekcje

```
section {
  background: #FFFFFF;
  border-radius: 10px;
```

```
padding: 1.25rem 1.5rem;
margin-bottom: 1.5rem;
border: 1px solid #E3E3EA;
}

h2 {
margin-top: 0;
font-size: 1.2rem;
border-bottom: 2px solid var(--kolor-glowny);
padding-bottom: 0.4rem;
}

.lista-umiejetnosci {
margin: 0;
padding-left: 1.2rem;
}
```

Trzy sekcje zamieniają się w białe karty na szarym tle. `margin-top: 0` przy `h2` jest ważne: bez tego domyślny margines nagłówek zrobiłby pustą przestrzeń na górze każdej karty.

---

## Krok 9 — formularz i stan błędu

```
.pole {
margin-bottom: 1rem;
}

label {
display: block;
font-weight: 600;
margin-bottom: 0.3rem;
}

input, select, textarea {
width: 100%;
padding: 0.55rem 0.7rem;
font: inherit;
border: 1px solid #C7C7D1;
border-radius: 6px;
background: #FFFFFF;
}

input:focus, select:focus, textarea:focus {
outline: 2px solid var(--kolor-glowny);
outline-offset: 1px;
border-color: var(--kolor-glowny);
}
```

```

textarea {
  resize: vertical;
}

/* pole z błędem – czerwona ramka dodawana przez JavaScript */
.pole--blad input,
.pole--blad select,
.pole--blad textarea {
  border-color: var(--kolor-blad);
}

.blad {
  margin: 0.3rem 0 0;
  min-height: 1.2rem;          /* miejsce zarezerwowane, układ nie
skacze */
  font-size: 0.875rem;
  color: var(--kolor-blad);
}

```

Cztery rzeczy warte uwagi:

- **`label { display: block; }`** — `<label>` jest domyślnie elementem liniowym i bez tego usiadzie w jednej linii z polem. To pierwsza rzecz, która się „psuje”, gdy formularz wygląda dziwnie.
- **`font: inherit`** — pola formularza mają własny, systemowy font i bez tego wyglądają jak wklejone z innej strony.
- **`min-height` na komunikacie błędu** — miejsce jest zarezerwowane, zanim pojawi się tekst. Bez tego cały formularz podskakuje przy każdym błędzie.
- **`.pole--blad`** — tej klasy nigdzie nie ma w HTML-u. Doda ją JavaScript w kroku 13, a CSS ma być na to gotowy wcześniej.

`textarea { resize: vertical; }` pozwala rozciągać pole tylko w dół — w poziomie rozjechałoby układ.

---

## Krok 10 — przycisk, podziękowanie, stopka

```

.przycisk {
  padding: 0.6rem 1.4rem;
  font: inherit;
  font-weight: 600;
  color: #FFFFFF;
}

```

```

background: var(--kolor-glowny);
border: none;
border-radius: 6px;
cursor: pointer;
transition: background 0.15s ease;
}

.przycisk:hover {
background: var(--kolor-ciemny);
}

.podziekowanie {
padding: 1rem 1.25rem;
background: #EFEBFC;
border-left: 4px solid var(--kolor-glowny);
border-radius: 6px;
}

.podziekowanie h3 {
margin-top: 0;
color: var(--kolor-glowny);
}

.stopka {
padding-top: 0.5rem;
padding-bottom: 2rem;
font-size: 0.9rem;
color: #5A5A66;
}

a {
color: var(--kolor-glowny);
text-decoration: none;
}

a:hover {
text-decoration: underline;
color: var(--kolor-ciemny);
}

```

Wymaganie mówi o **widocznym stanie** `:hover` na przycisku i linkach — to te dwie reguły z `:hover`. `transition` sprawia, że zmiana koloru jest płynna, a nie skokowa.

**Sprawdź:** strona wygląda skończona. Najedź kursorem na przycisk i na linki w stopce — muszą zareagować. Zacommituj:

```

git add zadania/mb02/style.css
git commit -m "Wizytówka: style, flexbox w nagłówku, stan błędu"

```

---

## Część 3 — JavaScript

Cały ten rozdział to plik `main.js`. Kolejność bloków jest taka sama jak w gotowym rozwiązaniu — dopisujesz je jeden pod drugim.

### Krok 11 — uchwytty do elementów

Początek pliku:

```
const formularz = document.querySelector("#formularz");
const podziekowanie = document.querySelector("#podziekowanie");
const podziekowanieTresc = document.querySelector("#podziekowanie-tresc");
```

Te trzy elementy będą potrzebne w kilku miejscach, więc wyszukujemy je **raz**, na starcie, zamiast wołać `querySelector` przy każdym użyciu.

**Sprawdź od razu:** dopisz na chwilę `console.log(formularz);`, odśwież stronę i zajrzyj do konsoli. Musi się wypisać element `<form>`. Jeśli widzisz `null`, to albo literówka w `id`, albo `<script>` trafił do `<head>`.

---

### Krok 12 — opis pól

Zamiast pisać cztery razy prawie ten sam kod, opisujemy pola danymi:

```
const POLA = [
  {
    id: "imie",
    pusteKomunikat: "Podaj imię i nazwisko.",
    sprawdz: wartosc =>
      wartosc.length < 3 ? "Imię i nazwisko musi mieć co najmniej 3 znaki." : "",
  },
  {
    id: "email",
    pusteKomunikat: "Podaj adres e-mail.",
    sprawdz: wartosc =>
      !wartosc.includes("@") || !wartosc.includes(".")
      ? "To nie wygląda na poprawny adres e-mail."
      : "",
  },
];
```

```

    },
    {
      id: "temat",
      pusteKomunikat: "Wybierz temat wiadomości.",
    },
    {
      id: "wiadomosc",
      pusteKomunikat: "Napisz wiadomość.",
      sprawdz: wartosc =>
        wartosc.length < 10 ? "Wiadomość musi mieć co najmniej 10
        znaków." : "",
    },
  ],
];

```

Każdy wpis ma id pola (to samo, co w HTML-u) i komunikat na wypadek, gdy pole jest puste. sprawdz jest **opcjonalna** — to dodatkowa reguła dla pola, które zostało wypełnione.

Umowa: sprawdz zwraca **pusty napis, gdy jest dobrze**, a treść błędu, gdy źle. Dzięki temu w jednym miejscu wystarczy if (komunikat === "").

temat nie ma sprawdz — dla listy wystarczy sprawdzić, czy coś wybrano.

Dodanie piątego pola do formularza to teraz dopisanie jednego wpisu w tej tablicy i jednego <div class="pole"> w HTML-u. Żadnej innej zmiany w kodzie.

## Krok 13 — pokazywanie błędu

```

function pokazBlad(id, komunikat) {
  const pole = document.querySelector(`#${id}`);
  const miejsceNaBlad = document.querySelector(`#blad-${id}`);

  miejsceNaBlad.textContent = komunikat;
  pole.closest(".pole").classList.toggle("pole--blad", komunikat !== "");
}

```

Jedna funkcja obsługuje oba kierunki: pusty komunikat **kasuje** błąd, niepusty go pokazuje.

- ``#blad-\${id}`` — tu widać, po co identyfikatory muszą pasować: z imie robi się blad-imie.

- ``textContent``, nie ``innerHTML`` — wstawiamy tekst, nie kod. Nawyk na całe życie: `innerHTML` z danymi od użytkownika to droga do ataku XSS.
- ``closest(".pole")`` — wędruje w górę drzewa DOM do najbliższego rodzica z tą klasą. Nie musimy wiedzieć, jak głęboko leży pole.
- ``classList.toggle(nazwa, warunek)`` — drugi argument decyduje: `true` dodaje klasę, `false` usuwa. Jedna linijka zamiast `if/else`.

---

## Krok 14 — sprawdzanie pojedynczego pola

```
function sprawdzPole(opis) {
  const wartosc = document.querySelector(`#${opis.id}`).value.trim();

  let komunikat = "";
  if (wartosc === "") {
    komunikat = opis.pusteKomunikat;
  } else if (opis.sprawdz) {
    komunikat = opis.sprawdz(wartosc);
  }

  pokazBlad(opis.id, komunikat);
  return komunikat === "";
}
```

Kolejność sprawdzeń jest celowa: najpierw pustka, potem reguła dodatkowa. Gdyby było odwrotnie, puste pole dostałoby komunikat „musi mieć co najmniej 3 znaki” zamiast „Podaj imię i nazwisko”.

``trim()`` jest tu **najważniejsze**. Bez niego pole wypełnione samymi spacjami przejdzie walidację — a to najczęściej wykorzystywana dziura w formularzach uczniowskich.

Funkcja robi dwie rzeczy naraz: **pokazuje** błąd i **zwraca** `true/false`. Dzięki temu w kroku 16 wystarczy jedno przejście po wszystkich polach.

---

## Krok 15 — poprawianie błędu w locie

```
POLA.forEach(opis => {
  const pole = document.querySelector(`#${opis.id}`);
  pole.addEventListener("input", () => {
    if (pole.closest(".pole").classList.contains("pole--blad")) {
      sprawdzPole(opis);
    }
  });
});
```

Gdy pole miało błąd i użytkownik zaczyna je poprawiać, komunikat znika od razu — nie trzeba czekać na kolejne kliknięcie „Wyślij”.

Warunek `if` jest tu istotny: sprawdzamy **tylko** pola, które już miały błąd. Bez niego czerwony komunikat wyskakiwałby po pierwszej wpisanej literze, zanim użytkownik zdąży cokolwiek napisać.

`input` odpala się przy **każdym** znaku — inaczej niż `change`, które czeka na wyjście z pola.

---

## Krok 16 — wysłanie formularza

Ostatni blok pliku:

```
formularz.addEventListener("submit", event => {
  // Bez tego przeglądarka przeładuje stronę i wszystko zniknie.
  event.preventDefault();

  const wyniki = POLA.map(sprawdzPole);
  const wszystkoPoprawne = wyniki.every(ok => ok);

  if (!wszystkoPoprawne) {
    // Ustawiamy kursor w pierwszym błędnym polu.
    const pierwszyBlad = POLA[wyniki.indexOf(false)];
    document.querySelector(`#${pierwszyBlad.id}`).focus();
    return;
  }

  const imie = document.querySelector("#imie").value.trim();
  const temat = document.querySelector("#temat");
  const wybranyTemat =
    temat.options[temat.selectedIndex].textContent;

  podziekowanieTresc.textContent =
```

```
`${imie}, Twoja wiadomość w sprawie „${wybranyTemat}” została wysłana.` +  
`Odpowiem najszybciej, jak to możliwe.`;  
  
// Formularz znika, na jego miejscu pojawia się podziękowanie.  
formularz.hidden = true;  
podziekowanie.hidden = false;  
});
```

``event.preventDefault()`` to linijka, bez której nic nie zadziała. Domyślnie submit przeładowuje stronę — formularz mrugnie i wróci pusty. Jeśli Twoja walidacja „nie działa”, zacznij szukać właśnie tutaj.

**Dlaczego ``map``, a nie ``find``?** `find` zatrzymałby się na pierwszym błędzie i użytkownik poprawiałby pola po kolei, jedno na raz. `map` sprawdza wszystkie, więc od razu widać komplet błędów. `.every(ok => ok)` zwraca `true` tylko wtedy, gdy **wszystkie** wyniki są prawdziwe.

**Dlaczego ``options[selectedIndex].textContent``, a nie ``temat.value``?** `value` da wartość techniczną ("wspolpraca"), a nam potrzebny napis widoczny dla człowieka ("Propozycja współpracy").

Zacommituj:

```
git add zadania/mb02/main.js  
git commit -m "Wizytówka: walidacja formularza i podziękowanie"
```

---